

# Smart Campus Management System

CS-104L Object-Oriented Programming

HITEC University Taxila

Date: 05-06-2026

GitHub URL: <https://github.com/ahmadoffcl/OOP>

## Group Members

- Ahmad Ali | Reg No: 25-CS-067
- Umer Altaf | Reg No: 25-CS-057
- Muhammed Ahmad | Reg No: 25-CS-252

## Introduction

This project is a simple command-line Smart Campus Management System written in C++. It manages basic campus records for students, faculty, courses, library items, fee records, hostel rooms, and reports. The purpose is to show all major OOP concepts from the CS-104L course in one connected program. The program uses simple arrays, pointers, classes, file handling, and exception handling. The design is intentionally easy to read so each group member can explain the code during viva.

## System Modules

- Person module: Person, Student, GradStudent, Faculty, and Staff classes.
- PersonManager: saved person records with add, view, delete, save, and reload options.
- Course module: Course, Enrollment, and CourseManager classes with saved course/enrollment records and capacity checking.
- Library module: Book and Journal catalog with add/search/delete, issue/return, overdue fine, file handling, and arrays.
- Finance module: FeeRecord and Invoice classes with copy handling and static invoice counter.
- Hostel module: Room, HostelBlock, and HostelManager using composition and multiple inheritance.
- Reports module: student sorting, searching, campus text report, and PDF-style text report generation.

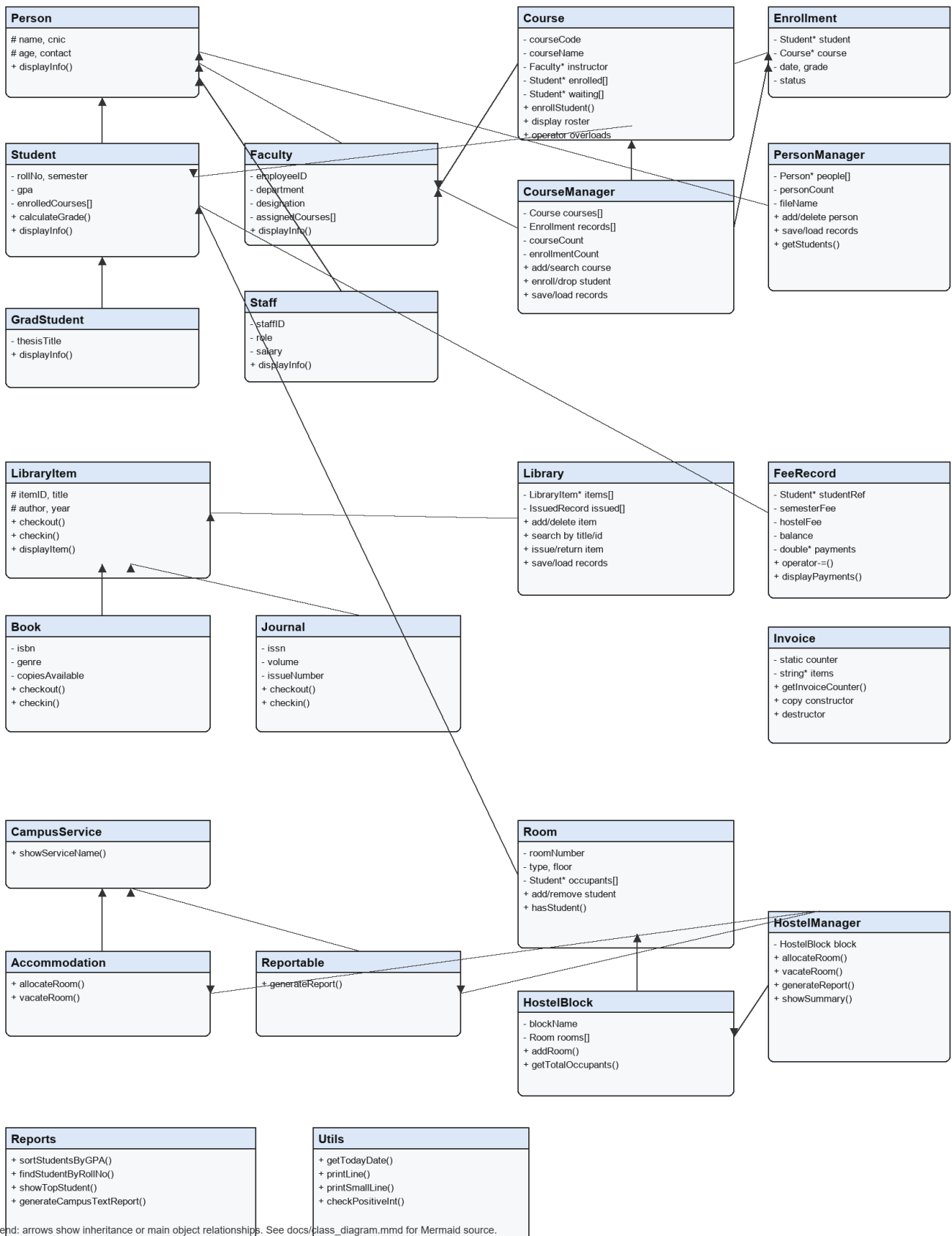
## Class Diagram

The class diagram below shows the main classes and relationships used in the project. Inheritance is used in Person, LibraryItem, and hostel interfaces. Composition is used where HostelManager contains HostelBlock, and aggregation is used where Course stores a Faculty pointer without owning the faculty object.

# Smart Campus Management System (SCMS)

## Smart Campus Management System - Class Diagram

HITEC University Taxila | CS-104L OOP



## OOP Concepts Implementation

1. Classes and Objects: All modules define classes and make objects in main.cpp.

```
Student s1(...); Course oop(...);
```

2. Encapsulation: Private data is changed through public getters, setters, and member functions.

```
string getRollNo() const; void setGPA(double newGPA);
```

3. Default Constructor: Several classes provide simple empty/default object creation.

```
Student::Student() : Person(), semester(1), gpa(0.0) {}
```

4. Parameterized Constructor: Objects can be created with real starting data.

```
Course::Course(string code, string name, int credits, Faculty* teacher, int capacity)
```

5. Copy Constructor: Person, Course, FeeRecord, and Invoice copy object data.

```
FeeRecord::FeeRecord(const FeeRecord& other)
```

6. Destructors: Library, Invoice, and HostelManager have destructors for cleanup.

```
Invoice::~~Invoice() { delete[] items; }
```

7. Single Inheritance: Student reuses common Person data and behavior.

```
class Student : public Person
```

8. Multi-level Inheritance: GradStudent extends Student, which already extends Person.

```
class GradStudent : public Student
```

9. Multiple Inheritance: HostelManager implements accommodation and report behavior.

```
class HostelManager : public Accommodation, public Reportable
```

10. Virtual Inheritance: Accommodation and Reportable virtually inherit CampusService to avoid duplicate base parts.

```
class Accommodation : virtual public CampusService
```

11. Abstract Classes: Person, LibraryItem, Accommodation, and Reportable cannot be directly instantiated.

```
virtual void displayInfo() const = 0;
```

12. Runtime Polymorphism: Base pointers call the correct derived displayInfo function.

```
Person* people[MAX_PEOPLE]; people[i]->displayInfo();
```

13. Operator Overloading ==: Course objects are compared by course code.

```
bool Course::operator==(const Course& other) const
```

14. Operator Overloading <<: Course and Invoice can be printed using stream insertion.

```
ostream& operator<<(ostream& out, const Course& course)
```

15. Operator Overloading +: Two course waiting lists are merged.

```
Student** Course::operator+(Course& other)
```

16. Operator Overloading -=: FeeRecord records a payment and reduces balance.

```
fee -= 25000;
```

17. Friend Functions: operator<< is declared as a friend for Course and Invoice.

```
friend ostream& operator<<(ostream& out, const Invoice& invoice);
```

18. Static Members: Invoice has one shared invoiceCounter for all invoice objects.

```
int Invoice::invoiceCounter = 0;
```

19. Deep Copy: FeeRecord and Invoice copy dynamic arrays instead of sharing them.

```
payments = new double[paymentCount];
```

## Smart Campus Management System (SCMS)

20. Search Functions: Library searches with a loop, and Reports uses `std::find_if` for roll number search.

```
Student* Reports::findStudentByRollNo(Student* students[], int count, string rollNo)
```

21. Array-based Collections: The project uses arrays for people, courses, library items, rooms, and students.

```
Person* people[MAX_PEOPLE];
```

22. Arrays of Objects: HostelBlock keeps Room objects in an array.

```
Room rooms[MAX_BLOCK_ROOMS];
```

23. Exception Handling: Custom exceptions are thrown and caught for capacity and overdue cases.

```
throw CapacityExceededException("Course is full");
```

24. File I/O: Person, course, enrollment, library, and report data are loaded/saved using `fstream`.

```
ofstream file(fileName);
```

25. Reporting and Utilities: Reports and Utils keep report, date, formatting, and validation helpers separate.

```
Reports::generateCampusTextReport(...);
```

26. Memory Management: Objects and arrays created with `new` are deleted using `delete` or `delete[]`.

```
delete[] sortedStudents;
```

27. Sorting and Searching: Reports sorts students by GPA and searches by roll number.

```
sortStudentsByGPA(students, count);
```

28. Composition: HostelManager owns a HostelBlock object.

```
HostelBlock block;
```

29. Aggregation: Course stores Faculty\* and Room stores Student\* without owning those people.

```
Faculty* instructor; Student* occupants[];
```

# Smart Campus Management System (SCMS)

## Module Descriptions

- Module 1 - Person Hierarchy: Stores common personal information in Person and uses Student, GradStudent, Faculty, and Staff subclasses. PersonManager stores saved records, supports input, deletion, save, and reload.
- Module 2 - Course and Enrollment: Stores saved course data, enrolls saved students, tracks waiting-list records, saves/reloads text files, blocks over-capacity enrollment, and demonstrates overloaded operators.
- Module 3 - Library System: Stores books and journals, adds/deletes catalog items, searches by title or ID, saves/loads catalog and issued records, tracks issued items, and handles overdue fines.
- Module 4 - Fee and Finance: Stores fee balance, records payments with operator=, deep-copies fee data, and generates invoices with a static counter.
- Module 5 - Hostel Management: Uses rooms, hostel blocks, multiple inheritance, virtual inheritance, and composition to allocate/vacate rooms.
- Module 6 - Reporting and Utilities: Sorts and searches student data, prints reports, writes a campus text report, writes a PDF-style text report, and keeps helper functions separate.

## GitHub Workflow

The project is already a local git repository with separate commits for setup, documentation, and each major module. Before final submission, create a public GitHub repository named HITEC-OOP-SCMS-25-CS-067, push this folder, and paste the repository URL here and in README.md.

- Local branch: master
- Current local commits: see git log --oneline in the repository.
- A GitHub Actions build workflow is included at .github/workflows/build.yml for the bonus compile check.
- Do not submit until the public GitHub link is shared through the course portal.

## Testing Summary

- Build command .\build.bat completed successfully.
- All six home menu modules were tested.
- Screen-based navigation was tested with Enter pauses after action pages.
- Module 1 CRUD was tested: add, display, save, reload, delete, save, and reload.
- Module 1 add-course action was tested by adding CS-200 to 25-CS-067.
- Module 2 added a course, searched it, enrolled students, sent a full-course student to waiting list, showed roster, dropped a student, saved/reloaded records, compared courses, and merged waiting lists.
- Library module added book/journal records, searched by title and ID, issued and returned items, blocked duplicate issue, showed overdue fine, deleted an item, and saved/reloaded records.
- Finance demo showed payment, copy constructor, copy assignment, invoice, and invoice copy.
- Hostel demo showed service name, allocation, duplicate check, summary, report, and vacate room.
- Reports demo sorted students by GPA and created data/campus\_report.txt.
- Reports demo created data/campus\_pdf\_report.txt.
- Wrong input test with abc did not freeze the program.
- Full captured output files are saved in docs/test\_outputs.

## Module1 Person Output Screenshot

```
Module 1 - Person Hierarchy

-----
SMART CAMPUS MANAGEMENT SYSTEM
-----
Group: Ahmad Ali, Umer Altaf, Muhammed Ahmad
Course: CS-104L Object-Oriented Programming
-----
1. Person Module
2. Course and Enrollment Module
3. Library Module
4. Fee and Finance Module
5. Hostel Module
6. Reports Module
0. Exit
Enter choice:
-----
PERSON MODULE
-----
1. Show all saved people
2. Show saved students
3. Add Student
4. Add Grad Student
5. Add Faculty
6. Add Staff
7. Add course to student
8. Delete person by ID
9. Save records
10. Reload records
0. Back to Home
Enter choice:
-----
SAVED PERSON RECORDS
-----

Student Name: Ahmad Ali
Roll No: 25-CS-067
Semester: 2
GPA: 3.4
Grade: B
Courses: No course added

... see module1_person.txt for full captured output ...
```

## Module1 Crud Test Output Screenshot

```
Module 1 - Person CRUD

-----
SMART CAMPUS MANAGEMENT SYSTEM
-----
Group: Ahmad Ali, Umer Altaf, Muhammed Ahmad
Course: CS-104L Object-Oriented Programming
-----
1. Person Module
2. Course and Enrollment Module
3. Library Module
4. Fee and Finance Module
5. Hostel Module
6. Reports Module
0. Exit
Enter choice:
-----
PERSON MODULE
-----
1. Show all saved people
2. Show saved students
3. Add Student
4. Add Grad Student
5. Add Faculty
6. Add Staff
7. Add course to student
8. Delete person by ID
9. Save records
10. Reload records
0. Back to Home
Enter choice:
-----
ADD STUDENT
-----
Name: CNIC: Age: Contact: Roll No: Semester: GPA: Student added. Use option 9 to save records.

Press Enter to continue...
-----
PERSON MODULE
-----
1. Show all saved people

... see module1_crud_test.txt for full captured output ...
```

## Module1 Add Course Test Output Screenshot

```
Module 1 - Add Course to Student

-----
SMART CAMPUS MANAGEMENT SYSTEM
-----
Group: Ahmad Ali, Umer Altaf, Muhammed Ahmad
Course: CS-104L Object-Oriented Programming
-----
1. Person Module
2. Course and Enrollment Module
3. Library Module
4. Fee and Finance Module
5. Hostel Module
6. Reports Module
0. Exit
Enter choice:
-----
PERSON MODULE
-----
1. Show all saved people
2. Show saved students
3. Add Student
4. Add Grad Student
5. Add Faculty
6. Add Staff
7. Add course to student
8. Delete person by ID
9. Save records
10. Reload records
0. Back to Home
Enter choice:
-----
ADD COURSE TO STUDENT
-----
Student Roll No: Course Code: Course added to student for this session.

Press Enter to continue...
-----
PERSON MODULE
-----
1. Show all saved people

... see module1_add_course_test.txt for full captured output ...
```



## Module2 Course Output Screenshot

```
Module 2 - Course and Enrollment

-----
SMART CAMPUS MANAGEMENT SYSTEM
-----
Group: Ahmad Ali, Umer Altaf, Muhammed Ahmad
Course: CS-104L Object-Oriented Programming
-----
1. Person Module
2. Course and Enrollment Module
3. Library Module
4. Fee and Finance Module
5. Hostel Module
6. Reports Module
0. Exit
Enter choice:
-----
COURSE AND ENROLLMENT MODULE
-----
1. Show all courses
2. Add course
3. Search course by code
4. Enroll student in course
5. Show course roster
6. Drop student from course
7. Compare two courses using ==
8. Merge waiting lists using +
9. Show enrollment records
10. Save course/enrollment records
11. Reload course/enrollment records
0. Back to Home
Enter choice:
-----
ADD COURSE
-----
Saved Faculty:
1. F-101 | Sir Usman | Computer Science
-----
Course Code: Course Name: Credit Hours: Max Capacity: Instructor Employee ID: Course added. Use option 10 to save r

Press Enter to continue...

... see module2_course.txt for full captured output ...
```

## Module3 Library Output Screenshot

```
Module 3 - Library System

-----
SMART CAMPUS MANAGEMENT SYSTEM
-----
Group: Ahmad Ali, Umer Altaf, Muhammed Ahmad
Course: CS-104L Object-Oriented Programming
-----
1. Person Module
2. Course and Enrollment Module
3. Library Module
4. Fee and Finance Module
5. Hostel Module
6. Reports Module
0. Exit
Enter choice:
-----
LIBRARY MODULE
-----
1. Show catalog
2. Add Book
3. Add Journal
4. Search by title
5. Search by item ID
6. Issue item
7. Return item
8. Show issued records
9. Delete item by ID
10. Save library records
11. Reload library records
0. Back to Home
Enter choice:
-----
LIBRARY CATALOG
-----

--- Library Catalog ---

Book ID: B001
Title: C++ Basics
Author: Deitel

... see module3_library.txt for full captured output ...
```

## Module4 Finance Output Screenshot

```
Module 4 - Fee and Finance

-----
SMART CAMPUS MANAGEMENT SYSTEM
-----
Group: Ahmad Ali, Umer Altaf, Muhammed Ahmad
Course: CS-104L Object-Oriented Programming
-----
1. Person Module
2. Course and Enrollment Module
3. Library Module
4. Fee and Finance Module
5. Hostel Module
6. Reports Module
0. Exit
Enter choice:

-----
FEE AND FINANCE MODULE
-----
1. Show fee record and payment
2. Show fee copy constructor and assignment
3. Show invoice, copies, and static counter
0. Back to Home
Enter choice:

-----
FEE RECORD DEMO
-----

--- Fee Record ---
Student: Ahmad Ali
Roll No: 25-CS-067
Semester Fee: 65000
Hostel Fee: 20000
Library Fine: 500
Total Paid: 0
Balance: 85500

After payment of 25000:

--- Fee Record ---
Student: Ahmad Ali

... see module4_finance.txt for full captured output ...
```

## Module5 Hostel Output Screenshot

```
Module 5 - Hostel Management

-----
SMART CAMPUS MANAGEMENT SYSTEM
-----
Group: Ahmad Ali, Umer Altaf, Muhammed Ahmad
Course: CS-104L Object-Oriented Programming
-----
1. Person Module
2. Course and Enrollment Module
3. Library Module
4. Fee and Finance Module
5. Hostel Module
6. Reports Module
0. Exit
Enter choice:
-----
HOSTEL MODULE
-----
1. Show hostel service name
2. Allocate sample students
3. Try duplicate allocation
4. Show hostel summary
5. Show occupancy report
6. Vacate first student room
0. Back to Home
Enter choice:
-----
HOSTEL SERVICE
-----
Hostel Accommodation Service

Press Enter to continue...
-----
HOSTEL MODULE
-----
1. Show hostel service name
2. Allocate sample students
3. Try duplicate allocation
4. Show hostel summary
5. Show occupancy report

... see module5_hostel.txt for full captured output ...
```

## Module6 Reports Output Screenshot

```
Module 6 - Reports and Utilities

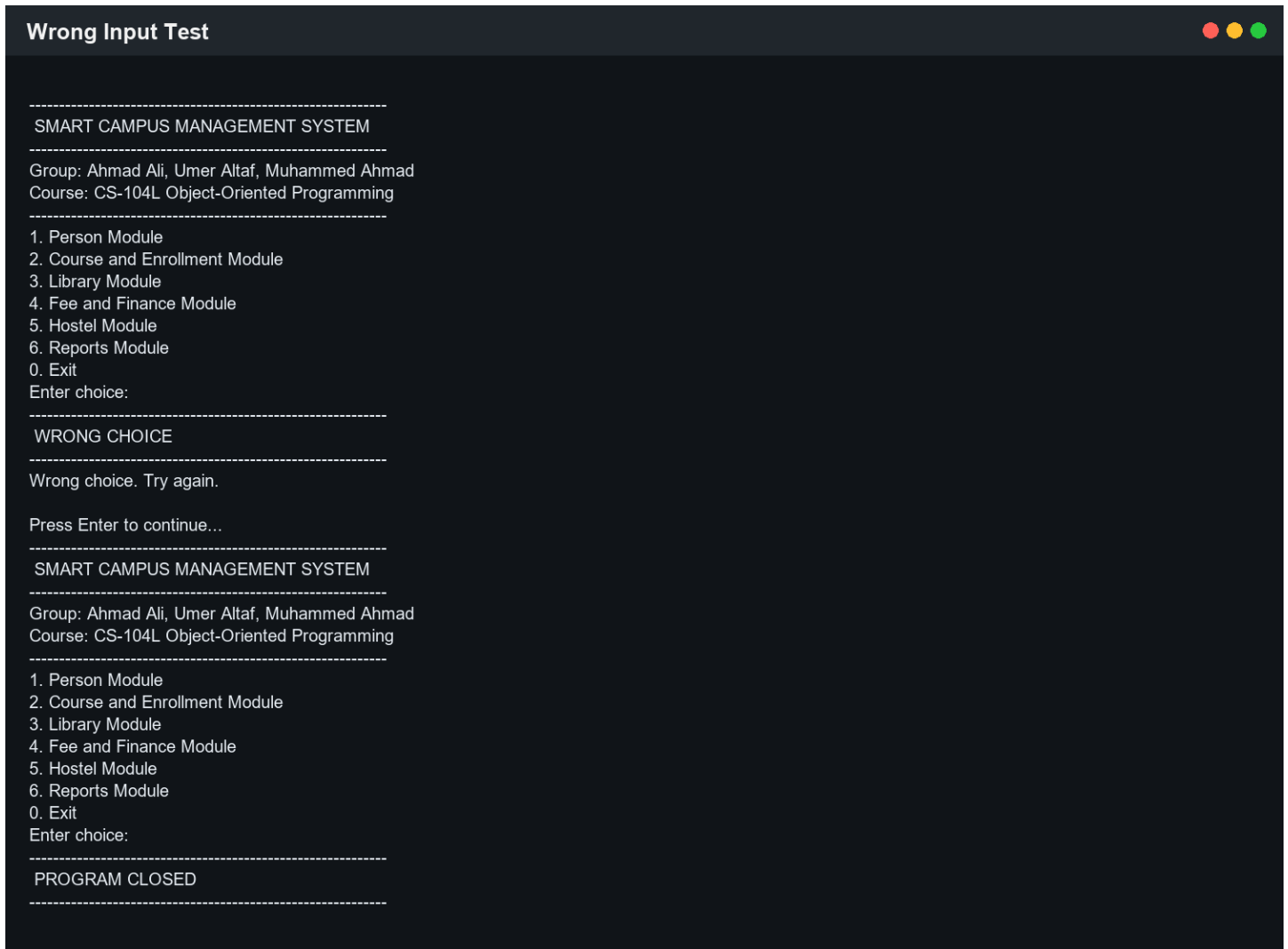
-----
SMART CAMPUS MANAGEMENT SYSTEM
-----
Group: Ahmad Ali, Umer Altaf, Muhammed Ahmad
Course: CS-104L Object-Oriented Programming
-----
1. Person Module
2. Course and Enrollment Module
3. Library Module
4. Fee and Finance Module
5. Hostel Module
6. Reports Module
0. Exit
Enter choice:
-----
REPORTS MODULE
-----
1. Sort and show students by GPA
2. Find student by roll number
3. Show top GPA student
4. Generate campus text report
5. Generate PDF-style text report
0. Back to Home
Enter choice:
-----
STUDENTS SORTED BY GPA
-----

--- Students Sorted by GPA ---
25-CS-057 - Umer Altaf - GPA: 3.8
25-CS-067 - Ahmad Ali - GPA: 3.4
25-CS-252 - Muhammed Ahmad - GPA: 3.2

Press Enter to continue...
-----
REPORTS MODULE
-----
1. Sort and show students by GPA
2. Find student by roll number

... see module6_reports.txt for full captured output ...
```

## Wrong Input Output Screenshot



### Challenges and Solutions

- Understanding polymorphism: solved by using Person\* array in main.cpp.
- Managing dynamic memory: solved by deleting Library items and Invoice item arrays.
- Handling full course capacity: solved by using a custom exception class.
- File handling format: solved by using simple pipe-separated text data.

### Conclusion

This project helped practice core C++ OOP concepts in a practical campus system. The project is intentionally console-based and simple so each module can be explained in viva. Before final submission, the group should push the repository to GitHub and replace the placeholder GitHub URL in this report.

### References

- Course project brief provided by HITEC University Taxila.
- C++ class notes and lab practice.
- cppreference.com for basic C++ syntax reference.